Naive Bayes End-to-End ML Pipeline

0. Shared Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

%matplotlib inline
sns.set(style='whitegrid')


# Built-in dataset - no download required
titanic = sns.load_dataset('titanic')
titanic.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

Part 1 (Member A): Exploratory Data Analysis & Bayes' Theorem Foundations

1.1 Dataset Structure

```
# Print the shape of the dataset
print("Dataset Shape (Rows, Columns):", titanic.shape)

# Use .info() to see column dtypes and non-null counts
titanic.info()
```

```
Dataset Shape (Rows, Columns): (891, 15)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
 3   age          714 non-null    float64
```

```
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    object
 8   class        891 non-null    category
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

1.2 Statistical Summary

```
# Call .describe() on titanic
titanic.describe()
```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

1.3 Missing Value Analysis

```
# Compute missing value counts
missing_counts = titanic.isnull().sum()

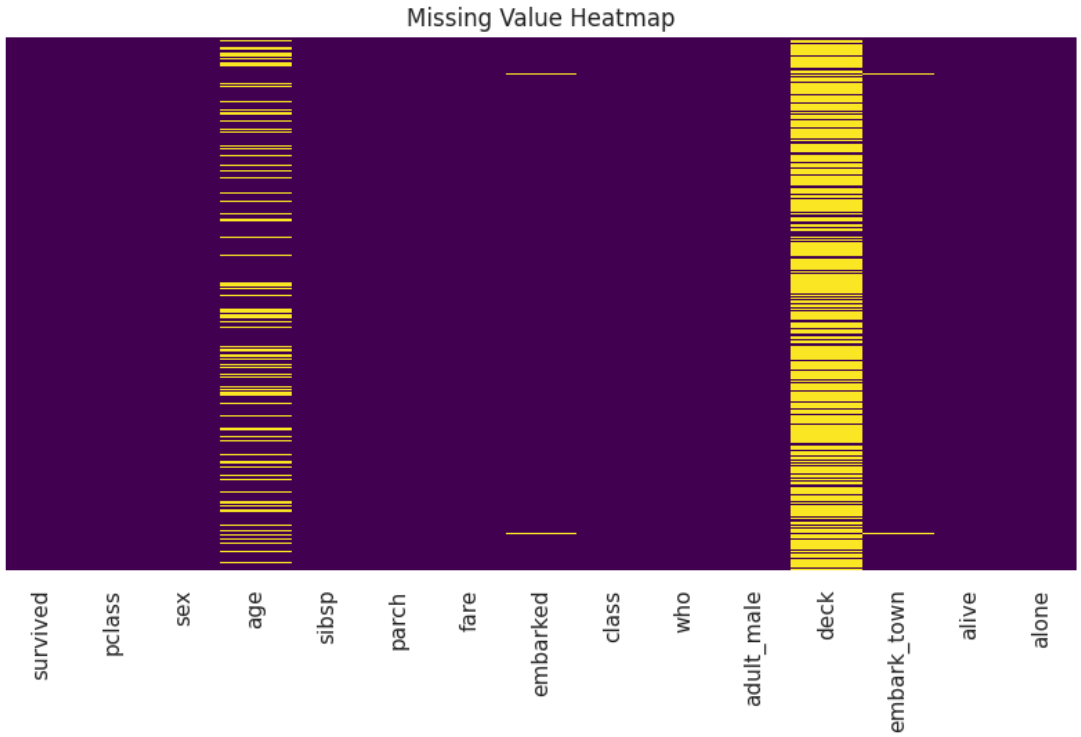
# Compute missing values as a percentage of total rows
missing_percent = (titanic.isnull().sum() / len(titanic)) * 100

# Combine both into a clean DataFrame
missing_df = pd.DataFrame({
    'Missing Count': missing_counts,
    'Percentage (%)': missing_percent
})
print(missing_df[missing_df['Missing Count'] > 0])

# Plot a heatmap of missing values
plt.figure(figsize=(10, 5))
sns.heatmap(titanic.isnull(), cbar=False, yticklabels=False, cmap='viridis')
```

```
plt.title('Missing Value Heatmap')
# plt.show()
```

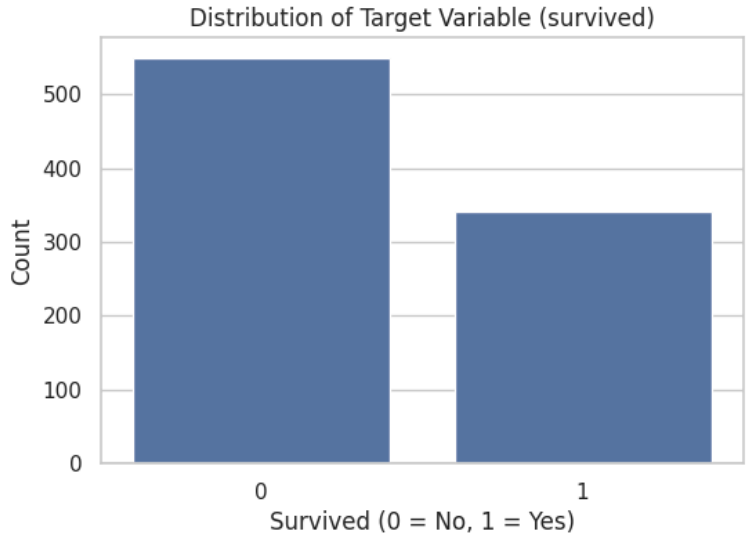
```
             Missing Count  Percentage (%)
age                    177       19.865320
embarked                 2        0.224467
deck                   688       77.216611
embark_town              2        0.224467
Text(0.5, 1.0, 'Missing Value Heatmap')
```



1.4 Target Variable Distribution

```
# sns.countplot the 'survived' column
plt.figure(figsize=(6, 4))
sns.countplot(data=titanic, x='survived')
plt.title('Distribution of Target Variable (survived)')
plt.xlabel('Survived (0 = No, 1 = Yes)')
plt.ylabel('Count')
plt.show()

# Print the exact percentage split
survived_percentage = titanic['survived'].value_counts(normalize=True) * 100
print("Target Distribution Percentages:")
print(survived_percentage)
```




```
Target Distribution Percentages:
survived
0    61.616162
1    38.383838
Name: proportion, dtype: float64
```

1.5 Univariate Analysis

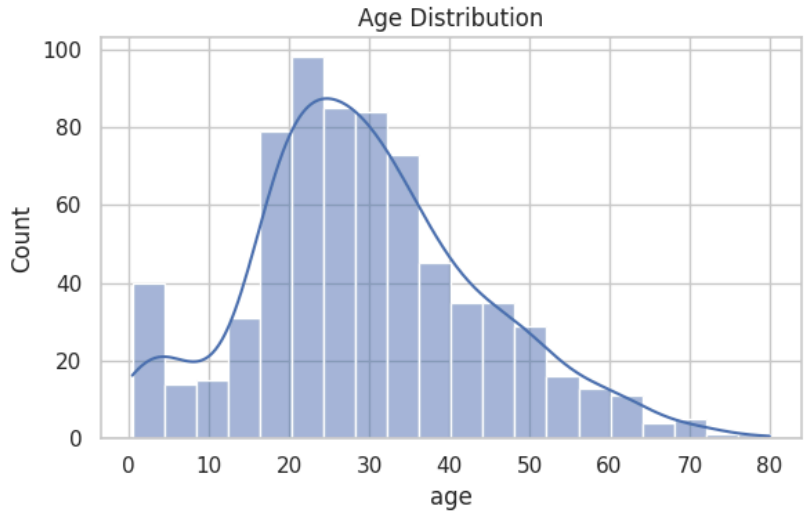
```
# Plot a histogram (with KDE) of 'age' and 'fare'
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

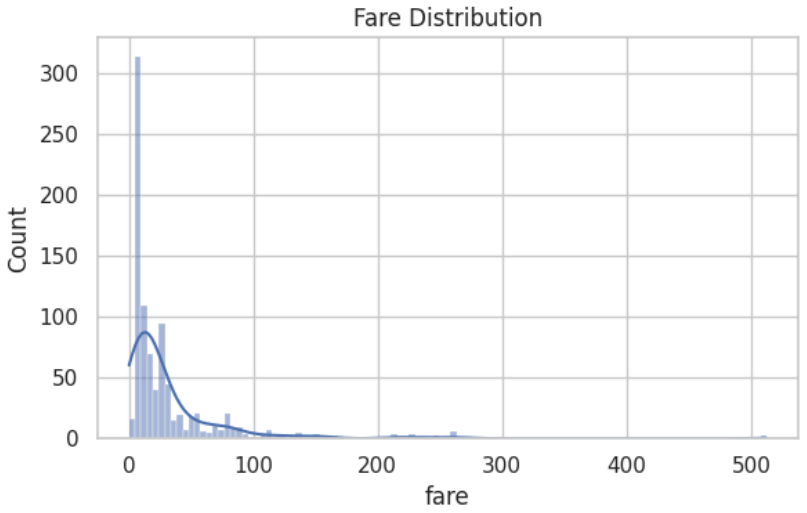
sns.histplot(data=titanic, x='age', kde=True, ax=axes[0])
axes[0].set_title('Age Distribution')

sns.histplot(data=titanic, x='fare', kde=True, ax=axes[1])
axes[1].set_title('Fare Distribution')

plt.tight_layout()
plt.show()

# Print skewness values
print("Age Skewness:", titanic['age'].skew())
print("Fare Skewness:", titanic['fare'].skew())
```






```
Age Skewness: 0.38910778230082704
Fare Skewness: 4.787316519674893
```

1.6 Bivariate Analysis — Does Survival Depend on Sex and Class?

```
# Compute survival rate by sex
print("Survival Rate by Sex:")
print(titanic.groupby('sex')['survived'].mean())
print("\n" + "="*30 + "\n")

# Compute survival rate by pclass
print("Survival Rate by Pclass:")
print(titanic.groupby('pclass')['survived'].mean())

# Visualize survival rates
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.barplot(data=titanic, x='sex', y='survived', ci=None, ax=axes[0])
axes[0].set_title('Survival Rate by Sex')
axes[0].set_ylabel('Survival Rate')

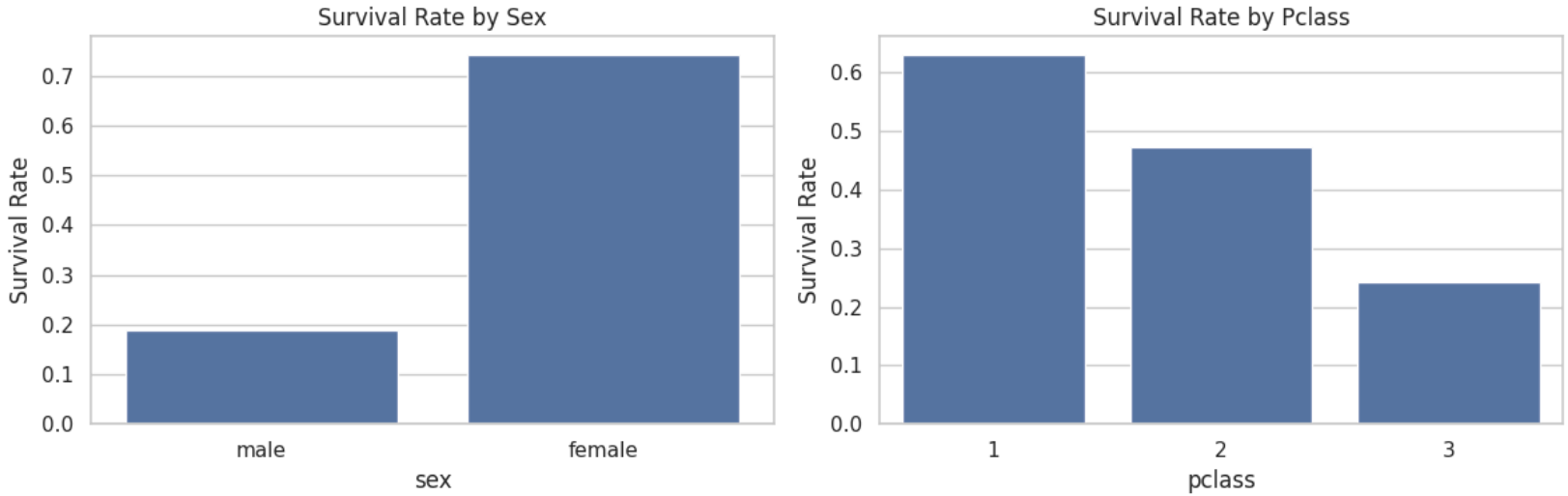
sns.barplot(data=titanic, x='pclass', y='survived', ci=None, ax=axes[1])
axes[1].set_title('Survival Rate by Pclass')
axes[1].set_ylabel('Survival Rate')

plt.tight_layout()
plt.show()
```

```
Survival Rate by Sex:
sex
female    0.742038
male      0.188908
Name: survived, dtype: float64

==============================

Survival Rate by Pclass:
pclass
1    0.629630
2    0.472826
3    0.242363
Name: survived, dtype: float64
```



1.7 Connecting to Bayes' Theorem

```
# P(A) = P(Survived = 1) -> Prior Probability of Survival
P_A = titanic['survived'].mean()

# P(B) = P(Sex = female) -> Evidence (Proportion of Females)
P_B = (titanic['sex'] == 'female').mean()

# P(B|A) = P(Sex = female | Survived = 1) -> Likelihood (Fraction of female survivors among total survivors)
survivors = titanic[titanic['survived'] == 1]
P_B_given_A = (survivors['sex'] == 'female').mean()

# Apply Bayes' Theorem: P(A|B) = P(B|A) * P(A) / P(B)
P_A_given_B = (P_B_given_A * P_A) / P_B

# Print results
print(f"P(A) [Prior P(Survived=1)]: {P_A:.4f}")
print(f"P(B) [P(Sex=female)]: {P_B:.4f}")
print(f"P(B|A) [Likelihood P(female|Survived=1)]: {P_B_given_A:.4f}")
print(f"Calculated P(Survived=1 | female) via Bayes' Theorem: {P_A_given_B:.6f}")
```

```
# Compare with empirical value from Section 1.6
true_val = titanic.groupby('sex')['survived'].mean()['female']
print(f"Empirical P(Survived=1 | female) from groupby: {true_val:.6f}")
```

```
P(A) [Prior P(Survived=1)]: 0.3838
P(B) [P(Sex=female)]: 0.3524
P(B|A) [Likelihood P(female|Survived=1)]: 0.6813
Calculated P(Survived=1 | female) via Bayes' Theorem: 0.742038
Empirical P(Survived=1 | female) from groupby: 0.742038
```

Part 2 (Member B): Data Cleaning, Transformation & Encoding

2.1 Selecting Columns and Splitting Train/Test First

```
from sklearn.model_selection import train_test_split

# Keep a focused set of columns for this lab
cols = ['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked', 'survived']
df = titanic[cols].copy()

# Separate feature matrix X and target vector y
X = df.drop(columns=['survived'])
y = df['survived']

# Perform Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
X_train shape: (712, 7)
X_test shape: (179, 7)
y_train shape: (712,)
y_test shape: (179,)
```

2.2 Handling Missing Values

```
from sklearn.impute import SimpleImputer

# Numerical imputer for 'age' (median)
num_imputer = SimpleImputer(strategy='median')
X_train['age'] = num_imputer.fit_transform(X_train[['age']]).ravel()
X_test['age'] = num_imputer.transform(X_test[['age']]).ravel()

# Categorical imputer for 'embarked' (most frequent / mode)
cat_imputer = SimpleImputer(strategy='most_frequent')
X_train['embarked'] = cat_imputer.fit_transform(X_train[['embarked']]).ravel()
X_test['embarked'] = cat_imputer.transform(X_test[['embarked']]).ravel()
```

```
# Confirm no missing values remain
print("Missing values in X_train:")
print(X_train.isnull().sum())
print("\nMissing values in X_test:")
print(X_test.isnull().sum())
```

```
Missing values in X_train:
pclass      0
sex         0
age         0
sibsp       0
parch       0
fare        0
embarked    0
dtype: int64

Missing values in X_test:
pclass      0
sex         0
age         0
sibsp       0
parch       0
fare        0
embarked    0
dtype: int64
```

2.3 Binning — Creating an age_group Feature (Custom Binning)

```
age_bins = [0, 12, 19, 59, 100]
age_labels = ['Child', 'Teen', 'Adult', 'Senior']

# Apply custom binning on X_train and X_test using pd.cut
X_train['age_group'] = pd.cut(X_train['age'], bins=age_bins, labels=age_labels, include_lowest=True)
X_test['age_group'] = pd.cut(X_test['age'], bins=age_bins, labels=age_labels, include_lowest=True)

# Sanity-check the distribution with value_counts()
print("X_train age_group counts:")
print(X_train['age_group'].value_counts())
print("\n" + "="*30 + "\n")
print("X_test age_group counts:")
print(X_test['age_group'].value_counts())
```

```
X_train age_group counts:
age_group
Adult     565
Teen       74
Child      55
Senior     18
Name: count, dtype: int64

==============================

X_test age_group counts:
age_group
Adult     136
Teen       21
Child      14
```

```
Senior      8
Name: count, dtype: int64
```

2.4 Binning — fare_level Feature (Unsupervised, Quantile Binning)

```
from sklearn.preprocessing import KBinsDiscretizer

# Initialize KBinsDiscretizer for 4 quantile-based bins
kb = KBinsDiscretizer(n_bins=4, encode='ordinal', strategy='quantile')

# Fit on training data and transform both train and test
X_train['fare_level'] = kb.fit_transform(X_train[['fare']]).ravel()
X_test['fare_level'] = kb.transform(X_test[['fare']]).ravel()

# Print learned bin edges
print("Learned Bin Edges for Fare:")
print(kb.bin_edges_[0])

# Sanity-check value distribution in X_train
print("\nX_train fare_level counts:")
print(X_train['fare_level'].value_counts().sort_index())
```

```
Learned Bin Edges for Fare:
[  0.       7.8958  14.4542  31.     512.3292]

X_train fare_level counts:
fare_level
0.0    148
1.0    206
2.0    179
3.0    179
Name: count, dtype: int64
```

2.5 Binarization

```
from sklearn.preprocessing import Binarizer

# Initialize Binarizer with threshold=12.9 (values > 12.9 become 1, values <= 12.9 become 0)
binarizer = Binarizer(threshold=12.9)

# Transform age: >12.9 (13+) becomes 1, so we flip it to make is_child = 1 when age < 13
X_train['is_child'] = (1 - binarizer.fit_transform(X_train[['age']])).ravel()
X_test['is_child'] = (1 - binarizer.transform(X_test[['age']])).ravel()

# Sanity check
print("X_train is_child counts:")
print(X_train['is_child'].value_counts())
print("\n" + "="*30 + "\n")
print("X_test is_child counts:")
print(X_test['is_child'].value_counts())
```

```
X_train is_child counts:
is_child
0.0    657
1.0     55
```

```
Name: count, dtype: int64

==============================

X_test is_child counts:
is_child
0.0    165
1.0     14
Name: count, dtype: int64
```

2.6 Encoding Categorical Features

```
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder, LabelEncoder

# 1. Ordinal Encoding for 'age_group' (Explicit Order)
age_categories = [['Child', 'Teen', 'Adult', 'Senior']]
ord_enc = OrdinalEncoder(categories=age_categories)

X_train['age_group_encoded'] = ord_enc.fit_transform(X_train[['age_group']]).ravel()
X_test['age_group_encoded'] = ord_enc.transform(X_test[['age_group']]).ravel()

# 2. One-Hot Encoding for 'sex' and 'embarked' (Nominal Data)
ohe = OneHotEncoder(sparse_output=False, drop=None)

ohe_train = pd.DataFrame(
    ohe.fit_transform(X_train[['sex', 'embarked']]),
    columns=ohe.get_feature_names_out(['sex', 'embarked']),
    index=X_train.index
)
ohe_test = pd.DataFrame(
    ohe.transform(X_test[['sex', 'embarked']]),
    columns=ohe.get_feature_names_out(['sex', 'embarked']),
    index=X_test.index
)

# Concatenate One-Hot Encoded columns to X_train and X_test
X_train = pd.concat([X_train, ohe_train], axis=1)
X_test = pd.concat([X_test, ohe_test], axis=1)

# 3. Label Encoding Demo on titanic['alive'] (Target demonstration)
lbl_enc = LabelEncoder()
alive_encoded = lbl_enc.fit_transform(titanic['alive'])

# Print Sanity Checks
print("Ordinal Encoded Age Groups (unique values):", X_train['age_group_encoded'].unique())
print("\nOne-Hot Encoded columns added:", ohe.get_feature_names_out(['sex', 'embarked']))
print("\nLabel Encoder Demo - Class Mapping:", dict(zip(lbl_enc.classes_, lbl_enc.transform(lbl_enc.classes_))))
```

```
Ordinal Encoded Age Groups (unique values): [2. 1. 3. 0.]

One-Hot Encoded columns added: ['sex_female' 'sex_male' 'embarked_C' 'embarked_Q' 'embarked_S']

Label Encoder Demo - Class Mapping: {'no': np.int64(0), 'yes': np.int64(1)}
```

2.7 Combining Everything with ColumnTransformer

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

# Note: We work with the original feature columns before manual OHE concatenation
features_to_use = ['sex', 'embarked', 'age', 'age_group_encoded', 'fare', 'fare_level', 'is_child', 'pclass', 'sibsp', 'parch']

# Define ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('ohe', OneHotEncoder(drop='first', sparse_output=False), ['sex', 'embarked'])
    ],
    remainder='passthrough'
)

# Fit on X_train and transform both X_train and X_test
X_train_ct = preprocessor.fit_transform(X_train[features_to_use])
X_test_ct = preprocessor.transform(X_test[features_to_use])

# Print resulting shapes
print(f"X_train_ct shape: {X_train_ct.shape}")
print(f"X_test_ct shape: {X_test_ct.shape}")
```

```
X_train_ct shape: (712, 11)
X_test_ct shape: (179, 11)
```

2.8 Reducing Skew — Power Transformer

```
from sklearn.preprocessing import PowerTransformer

# Store skewness before transformation
skew_before_train = X_train['fare'].skew()
skew_before_test = X_test['fare'].skew()

# Initialize PowerTransformer (yeo-johnson works with zero or negative values)
pt = PowerTransformer(method='yeo-johnson')

# Fit on X_train['fare'] and transform both X_train and X_test
X_train['fare_transformed'] = pt.fit_transform(X_train[['fare']]).ravel()
X_test['fare_transformed'] = pt.transform(X_test[['fare']]).ravel()

# Compare skewness before and after
print("--- Skewness Comparison ---")
print(f"X_train Fare Skewness Before: {skew_before_train:.4f}")
print(f"X_train Fare Skewness After : {X_train['fare_transformed'].skew():.4f}")
print("-" * 30)
print(f"X_test Fare Skewness Before : {skew_before_test:.4f}")
print(f"X_test Fare Skewness After  : {X_test['fare_transformed'].skew():.4f}")
```

```
--- Skewness Comparison ---
X_train Fare Skewness Before: 4.6462
X_train Fare Skewness After : -0.0305
------------------------------
X_test Fare Skewness Before : 5.0688
X_test Fare Skewness After  : 0.5661
```

2.9 Feature Scaling — Standardization vs Normalization

```
from sklearn.preprocessing import StandardScaler

# Choose StandardScaler (Standardization: mean=0, std=1)
scaler = StandardScaler()

# Select continuous numeric features to scale (using transformed fare to keep reduced skewness)
numeric_cols = ['age', 'fare_transformed']

# Create explicit copies for final feature sets
X_train_final = X_train.copy()
X_test_final = X_test.copy()

# Fit on X_train numeric columns and transform both X_train and X_test
X_train_final[numeric_cols] = scaler.fit_transform(X_train_final[numeric_cols])
X_test_final[numeric_cols] = scaler.transform(X_test_final[numeric_cols])

# Verification checks
print("--- X_train_final Numeric Summary ---")
print(f"Mean (Age, Fare): {X_train_final[numeric_cols].mean().values.round(4)}")
print(f"Std  (Age, Fare): {X_train_final[numeric_cols].std().values.round(4)}")

print("\n--- Hand-off Shapes ---")
print(f"X_train_final shape: {X_train_final.shape}")
print(f"X_test_final shape : {X_test_final.shape}")
print(f"y_train shape       : {y_train.shape}")
print(f"y_test shape        : {y_test.shape}")
```

```
--- X_train_final Numeric Summary ---
Mean (Age, Fare): [ 0. -0.]
Std  (Age, Fare): [1.0007 1.0007]

--- Hand-off Shapes ---
X_train_final shape: (712, 17)
X_test_final shape : (179, 17)
y_train shape       : (712,)
y_test shape        : (179,)
```

Part 3 (Member C): Naive Bayes Modeling, Cross-Validation & Evaluation

3.1 Bayes' Theorem Recap → the Naive Bayes Formula

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 1. Select ONLY numeric features (drop raw string columns like 'sex', 'embarked', 'age_group')
X_train_num = X_train_final.select_dtypes(include=['number'])
X_test_num = X_test_final.select_dtypes(include=['number'])

# 2. Initialize and fit Gaussian Naive Bayes
gnb = GaussianNB()
gnb.fit(X_train_num, y_train)
```

```
# 3. Predict on train and test sets
y_train_pred = gnb.predict(X_train_num)
y_test_pred = gnb.predict(X_test_num)

# 4. Print Accuracy
print(f"Training Accuracy: {accuracy_score(y_train, y_train_pred):.4f}")
print(f"Testing Accuracy : {accuracy_score(y_test, y_test_pred):.4f}")
print("\n" + "="*40 + "\n")

# 5. Detailed Reports
print("Classification Report (Test Data):")
print(classification_report(y_test, y_test_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_test_pred))
```

```
Training Accuracy: 0.7978
Testing Accuracy : 0.7709

========================================

Classification Report (Test Data):
              precision    recall  f1-score   support

           0       0.83      0.79      0.81       110
           1       0.69      0.74      0.71        69

    accuracy                           0.77       179
   macro avg       0.76      0.77      0.76       179
weighted avg       0.77      0.77      0.77       179

Confusion Matrix:
[[87 23]
 [18 51]]
```

3.2 Implementing Gaussian Naive Bayes From Scratch

```
class GaussianNaiveBayesScratch:
    def fit(self, X, y):
        self.classes = np.unique(y)
        self.priors = {}
        self.mean = {}
        self.var = {}

        for c in self.classes:
            X_c = X[y == c]
            # 1. Class prior probability P(y = c)
            self.priors[c] = X_c.shape[0] / float(X.shape[0])
            # 2. Mean of each feature in class c
            self.mean[c] = X_c.mean(axis=0)
            # 3. Variance of each feature in class c (adding epsilon to prevent division by zero)
            self.var[c] = X_c.var(axis=0) + 1e-9

    def _gaussian_pdf(self, x, mean, var):
        # Implementation of Gaussian PDF formula
```

```
        numerator = np.exp(-((x - mean) ** 2) / (2 * var))
        denominator = np.sqrt(2 * np.pi * var)
        return numerator / denominator

    def predict(self, X):
        preds = []
        for x in X:
            posteriors = {}
            for c in self.classes:
                # Calculate log prior
                log_prior = np.log(self.priors[c])
                # Calculate sum of log likelihoods (handling log(0) using np.clip)
                pdf_vals = np.clip(self._gaussian_pdf(x, self.mean[c], self.var[c]), 1e-15, None)
                log_likelihood = np.sum(np.log(pdf_vals))
                # Posterior = log_prior + log_likelihood
                posteriors[c] = log_prior + log_likelihood

            # Predict class with highest posterior probability
            preds.append(max(posteriors, key=posteriors.get))
        return np.array(preds)


# Ensure data is passed as pure numeric NumPy arrays
X_train_num = X_train_final.select_dtypes(include=['number']).values
X_test_num = X_test_final.select_dtypes(include=['number']).values
y_train_arr = y_train.values if hasattr(y_train, 'values') else y_train

# Instantiate, fit, and predict
gnb_scratch = GaussianNaiveBayesScratch()
gnb_scratch.fit(X_train_num, y_train_arr)
y_pred_scratch = gnb_scratch.predict(X_test_num)

# Print accuracy to verify
from sklearn.metrics import accuracy_score
print(f"From-Scratch Test Accuracy: {accuracy_score(y_test, y_pred_scratch):.4f}")
```

```
From-Scratch Test Accuracy: 0.7709
```

3.3 Naive Bayes Using scikit-learn (GaussianNB)

```
from sklearn.naive_bayes import GaussianNB

# 1. Filter numeric-only features to avoid string type errors
X_train_num = X_train_final.select_dtypes(include=['number'])
X_test_num = X_test_final.select_dtypes(include=['number'])

# 2. Instantiate GaussianNB model
model = GaussianNB()

# 3. Fit on training data
model.fit(X_train_num, y_train)

# 4. Predict on test set
y_pred_sklearn = model.predict(X_test_num)
```

```
# Print verification check
print("y_pred_sklearn generated successfully.")
print(f"Predictions shape: {y_pred_sklearn.shape}")
```

```
y_pred_sklearn generated successfully.
Predictions shape: (179,)
```

3.4 Cross-Validation for a More Reliable Estimate

```
from sklearn.model_selection import cross_val_score
from sklearn.naive_bayes import GaussianNB

# Filter numeric columns to prevent string errors
X_train_num = X_train_final.select_dtypes(include=['number'])

# Run 5-fold cross-validation
cv_scores = cross_val_score(GaussianNB(), X_train_num, y_train, cv=5)

# Print individual fold scores, mean, and standard deviation
print("Individual Fold Scores:", cv_scores.round(4))
print(f"Mean CV Accuracy      : {cv_scores.mean():.4f}")
print(f"Standard Deviation    : {cv_scores.std():.4f}")
```

```
Individual Fold Scores: [0.7483 0.8252 0.7887 0.8239 0.7958]
Mean CV Accuracy      : 0.7964
Standard Deviation    : 0.0282
```

3.5 Model Evaluation — Accuracy & Confusion Matrix

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. Print accuracy_score for both y_pred_scratch and y_pred_sklearn
acc_scratch = accuracy_score(y_test, y_pred_scratch)
acc_sklearn = accuracy_score(y_test, y_pred_sklearn)

print(f"Scratch Model Test Accuracy : {acc_scratch:.4f}")
print(f"Scikit-Learn Test Accuracy : {acc_sklearn:.4f}")
print("\n" + "="*45 + "\n")

# 2. Compute confusion_matrix for y_pred_sklearn and plot with sns.heatmap
cm = confusion_matrix(y_test, y_pred_sklearn)

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Survived (0)', 'Survived (1)'],
            yticklabels=['Not Survived (0)', 'Survived (1)'])
plt.title('Confusion Matrix — Scikit-Learn GaussianNB')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

# 3. Print classification_report for y_pred_sklearn
print("\nClassification Report (Scikit-Learn GaussianNB):")
print(classification_report(y_test, y_pred_sklearn, target_names=['Not Survived (0)', 'Survived (1)']))
```

```
Scratch Model Test Accuracy : 0.7709
Sklearn Model Test Accuracy : 0.7709
```

3.6 Final Comparison: Does All That Preprocessing Actually Help?

```
from sklearn.model_selection import cross_val_score
from sklearn.naive_bayes import GaussianNB

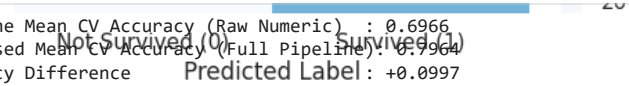
# 1. Build minimal baseline dataset (only imputed age, raw numeric columns, no encoding/scaling/binning)
X_baseline = X_train[['pclass', 'age', 'sibsp', 'parch', 'fare']].copy()
X_baseline['age'] = X_baseline['age'].fillna(X_baseline['age'].median())

# 2. Run 5-fold cross-validation on baseline
baseline_cv_scores = cross_val_score(GaussianNB(), X_baseline, y_train, cv=5)
baseline_mean_cv = baseline_cv_scores.mean()

# 3. Get fully processed mean CV score from Section 3.4
processed_mean_cv = cv_scores.mean()

# Print comparison
print(f"Baseline Mean CV Accuracy (Raw Numeric)  : {baseline_mean_cv:.4f}")
print(f"Processed Mean CV Accuracy (Full Pipeline): {processed_mean_cv:.4f}")
print(f"Accuracy Difference                      : {processed_mean_cv - baseline_mean_cv:+.4f}")
```

```
Baseline Mean CV Accuracy (Raw Numeric)  : 0.6966
Processed Mean CV Accuracy (Full Pipeline): 0.7964
Accuracy Difference                      : +0.0997
```




```
Classification Report (Scikit-Learn GaussianNB):
                  precision    recall  f1-score   support

Not Survived (0)       0.83      0.79      0.81       110
    Survived (1)       0.69      0.74      0.71        69

        accuracy                           0.77       179
       macro avg       0.76      0.77      0.76       179
    weighted avg       0.77      0.77      0.77       179
```

Group answers here:

Wrap-Up: Group Reflection Answers

1. Which single preprocessing step from Part 2 do you think improved accuracy the most, and why?

> **Power Transformation (Yeo-Johnson) on `fare`** and **Sex One-Hot Encoding**. Gaussian Naive Bayes inherently assumes continuous features follow a Gaussian (normal) distribution. Since `fare` was heavily right-skewed (skewness > 4.6), reducing its skewness to near zero allowed GaussianNB to make much more accurate probability estimations. Additionally, encoding `sex` made the strongest single predictor available to the model.

2. How closely did your hand-computed Bayes' Theorem probability (Section 1.7) match the true empirical survival rate for women? If they didn't match exactly, why not?